# Kubernetes Cluster Inventory Report

**Generated:** 2026-02-23 **Cluster:** DigitalOcean-hosted (registry: `registry.emrbear.com`)

---

## 1. Namespace Inventory

**Total namespaces: 183**

| Category | Namespaces | Count |
|---|---|---|
| Kubernetes system | `kube-system`, `kube-public`, `kube-node-lease` | 3 |
| Platform / infrastructure | `nginx-ingress`, `monitoring`, `velero`, `bear-deployer`, `bear-stats`, `bear-cron`, `services`, `sentry`, `kubecost` | 9 |
| Application: EMR Bear (tenant apps) | `acbcounseling`, `achieve`, `achievement`, `achr`, `acoma`, `acorn`, … (all `*.emrbear.com` tenants) | ~158 |
| Special / ancillary | `gemini`, `prex`, `default`, `staging`, `demo5`, `demo6`, `demotat` | 7+ |

All namespaces are `Active`. Oldest tenants date to ~6 years ago; newest (`discipleship`, `ebony`, `roadrunner`) are 27–44 days old.

---

## 2. Workload Inventory

**Summary:**

| Kind | Count |
|---|---|
| Deployments | 430 |
| StatefulSets | 4 |
| DaemonSets | 10 |
| CronJobs | 6 |
| Jobs | 14 |
| **Running Pods** | **1,141** |

### Standard Tenant Pattern (per ~167 tenant namespaces)

Each tenant namespace runs a uniform two-deployment pair:

| Deployment | Image | Purpose |
|---|---|---|
| `<tenant>-bear-app` | `registry.emrbear.com/bear5:<branch>-<sha>` | Web (Puma/Rails), 1–2 replicas |
| `delayed-<tenant>-bear-app-0` | `registry.emrbear.com/bear5:<branch>-<sha>` | Background worker (DelayedJob), 1 replica |

Two image tracks are in use:

- `master-7b4fda` — production (majority of tenants)
- `develop-e4f2be` — develop/staging track (e.g., `acbcounseling` )

### Infrastructure / Platform Workloads

| Namespace | Workload | Kind | Image | Purpose |
|---|---|---|---|---|
| `bear-deployer` | `bear-deployer` | Deployment | `bear5:master-af4334` | Deployment orchestration UI (currently scaled to 0) |
| `bear-deployer` | `bear-deployer-sidekiq` | Deployment | `bear5:master-af4334` | Sidekiq worker for deployer (scaled to 0) |
| `bear-deployer` | `redis-bear-deployer-master` | StatefulSet | `bitnami/redis:4.0.14` | Redis for bear-deployer Sidekiq queue |
| `bear-stats` | `bear-stats` | Deployment | `bear-stats:master-38c3fa` | Dashboard/stats app (`dash.emrbear.com`) |
| `bear-stats` | `bear-github` | Deployment | `bear-github:main-83d533` | GitHub API proxy service |
| `bear-cron` | `bear-daily-job` | CronJob | `bear-cron:latest` | Daily maintenance tasks (07:00 UTC) |
| `bear-cron` | `bear-hourly-job` | CronJob | `bear-cron:latest` | Hourly scheduled tasks |
| `services` | `bear-services` | Deployment | `bear-services:master-f39a206` + `bear-services-hds:latest` | Shared services API (`services.emrbear.com`), 2 containers |
| `services` | `bear-services-nginx` | Deployment | `nginx:alpine` | Nginx reverse proxy for services |
| `services` | `bear-xero` | Deployment | `bear-xero:master-13971d` | Xero accounting API connector |
| `services` | `bear-x12` | Deployment | `bear_x12` | X12 EDI processor (2 replicas) |
| `services` | `docker-registry` | Deployment | `registry:2.7.1` | Private Docker registry (backed by DO Spaces S3) |
| `services` | `trip-form-filler` | Deployment | `trip-reporter:1.1.0` | Trip form automation (3 replicas) |
| `gemini` | `staging-gemini-web` | Deployment | `bear-gemini:main-493aff` | Gemini staging web |

| | | | | |
|---|---|---|---|---|
| gemini | staging-gemini-worker | Deployment | bear-gemini:main-493aff | Gemini staging worker |
| gemini | production-gemini-web | Deployment | bear5:master-af4334 | Gemini production (currently scaled to 0) |
| default | fallacious-bronco-memcached | StatefulSet | Memcached | Shared Memcached (0+1 indexed pods) |
| monitoring | kube-prometheus-grafana | Deployment | grafana:9.2.4 | Grafana dashboards |
| monitoring | kube-prometheus-kube-prome-operator | Deployment | prometheus-operator:v0.60.1 | Prometheus Operator |
| monitoring | kube-prometheus-kube-state-metrics | Deployment | kube-state-metrics:v2.6.0 | K8s metrics exporter |
| monitoring | mysql-exporter-m3 | Deployment | mysqld-exporter:v0.15.0 | MySQL metrics exporter (instance: database_m3) |
| monitoring | prometheus-* | StatefulSet | prometheus:v2.39.1 | Prometheus metrics storage (50Gi PVC) |
| monitoring | alertmanager-* | StatefulSet | alertmanager:v0.24.0 | Alert routing |
| monitoring | kube-prometheus-prometheus-node-exporter | DaemonSet | node-exporter:v1.3.1 | Node metrics (79 nodes) |
| velero | velero | Deployment | velero:v1.10.0 | Cluster backup (reads AWS/Azure creds from cloud file) |

### Prex Namespace (Healthcare integrations)

| Workload | Image | Purpose |
|---|---|---|
| cerner-web (2 replicas) | prex-cerner:270f6297 | Cerner EHR integration |
| production-idology-web | prex-idology:c135cf59 | IDology identity verification (prod) |
| staging-idology-web | prex-idology:c135cf59 | IDology identity verification (staging) |
| production-surescripts-web (2 replicas) | prex-surescripts:97c1d743 | Surescripts e-prescribing (prod) |

| | | |
|---|---|---|
| `production-surescripts-worker` | `prex-surescripts:97c1d743` | Surescripts async worker (prod) |
| `staging-surescripts-web` | `prex-surescripts:97c1d743` | Surescripts (staging) |
| `staging-surescripts-worker` | `prex-surescripts:97c1d743` | Surescripts async worker (staging) |
| `saaspass-web` | `prex-saaspass:df08cf5a` | SaaSPass SSO integration |

---

## 3. Service & Database Dependency Map

### Database Architecture

All primary databases are **external to the cluster** (hosted on separate DigitalOcean droplets):

| Database | Type | How accessed |
|---|---|---|
| MySQL (per-tenant) | MySQL/MariaDB | Via `<tenant>-bear-db-secrets` → `database.yml` + `.my.cnf` mounted as secret |
| MongoDB | MongoDB | `mongodb.default.svc.cluster.local:27017` (DNS resolves to external droplet via cluster DNS) — used by `bear-services` |

**MySQL connection pattern per tenant:**

- Secret: `<tenant>-bear-db-secrets` containing `database.yml` and `.my.cnf`
- Credentials are file-mounted (not env vars), explaining why no `DATABASE_URL` env vars appear in the manifest scan

**In-cluster data stores:**

| Store | Location | Used By |
|---|---|---|
| Redis 4.0.14 | `bear-deployer/redis-bear-deployer-master:6379` | `bear-deployer` + `bear-deployer-sidekiq` (via `REDIS_URL` secret) |
| Memcached | `default/fallacious-bronco-memcached-{0,1}:11211` | All tenant `bear-app` pods (env `MEMCACHE_SERVERS` set directly) |

**Memcached connection string (all tenants):**

```
fallacious-bronco-memcached-0.fallacious-bronco-memcached.default.svc.cluster.local:11211:1,
fallacious-bronco-memcached-1.fallacious-bronco-memcached.default.svc.cluster.local:11211:2
```

### Internal Service Map (key services)

| Service | Namespace | Type | Port | Consumers |
|---|---|---|---|---|
| `<tenant>-bear-app` | `<tenant>` | ClusterIP | 8080 | nginx-ingress controller |

| | | | | |
|---|---|---|---|---|
| `maintenance-page` | `<tenant>` | ClusterIP | 80 | ingress (failover) |
| `redis-bear-deployer-master` | `bear-deployer` | ClusterIP | 6379 | bear-deployer pods |
| `redis-bear-deployer-headless` | `bear-deployer` | ClusterIP (None) | 6379 | Redis internal replication |
| `bear-deployer-web` | `bear-deployer` | ClusterIP | 4567 | ingress → `bear-deployer.emrbear.com` |
| `bear-stats` | `bear-stats` | ClusterIP | 3000 | ingress → `dash.emrbear.com` |
| `bear-github-api` | `bear-stats` | ClusterIP | 80 | bear-stats |

---

## 4. External Integrations

### Third-Party SaaS / APIs

| Integration | Workload | Namespace | Secret/Config |
|---|---|---|---|
| **Xero** (accounting) | `bear-xero` | `services` | `bear-xero-secrets` (consumer key + secret for NM and DE tenants) + `xero-config` ConfigMap (redirect URI, scopes) |
| **Sentry** (error tracking) | All `bear-app` pods + `bear-deployer` + `bear-services` | cluster-wide | `shared-runtime-secrets.SENTRY_DSN` (self-hosted: `sentry.emrbear.com`) |
| **AWS IoT** | All `bear-app` pods | cluster-wide | `shared-runtime-secrets.AWS_IOT_ENDPOINT` |
| **Gemini** (internal service?) | All `bear-app` pods | cluster-wide | `shared-runtime-secrets.BEAR_GEMINI_URL` |
| **S3 / DigitalOcean Spaces** | `docker-registry` | `services` | `registry-secret` — bucket: `registry-store-4`, region: `sfo3`, endpoint: `sfo3.digitaloceanspaces.com` |
| **Cerner EHR** | `cerner-web` | `prex` | `cerner-secrets`, `prex-aws-secrets`, `prex-postgres-secrets` |
| **IDology** (identity) | `*-idology-web` | `prex` | `idology-secrets` |
| **Surescripts** (e-prescribing) | `*-surescripts-*` | `prex` | `surescripts-secrets`, `prex-aws-secrets`, `prex-postgres-secrets` |
| **SaaSPass** (SSO) | `saaspass-web` | `prex` | `saaspass-secrets`, `prex-postgres-secrets` |
| **GitHub API** | `bear-github` | `bear-stats` | `bear-github-secrets` |

| | | | |
|---|---|---|---|
| **Velero backup** | `velero` | `velero` | Cloud credentials file at `/credentials/cloud` (AWS + Azure) |

### ExternalName Services

None found — external services are reached by hostname (DNS/env vars), not via Kubernetes ExternalName service objects.

### Persistent Storage (PVCs)

| Namespace | PVC | Size | Storage Class | Purpose |
|---|---|---|---|---|
| `hogares` | `csi-pvc-export-csi-holder-0` | 200Gi | `do-block-storage` | Legacy data export volume |
| `monitoring` | `prometheus-*-db` | 50Gi | `do-block-storage` | Prometheus TSDB |
| `sentry` | `redis-data-sentry-sentry-redis-{master,slave-0,slave-1}` | 8Gi × 3 | `do-block-storage` | Sentry Redis (no active pods — likely decommissioned, PVCs orphaned) |

---

## 5. Ingress & Networking

### Ingress Controller

Two nginx-ingress controllers are deployed in `nginx-ingress`:

| Controller | Service | NodePorts | Status |
|---|---|---|---|
| `nginx-ingress-controller-manual` (legacy) | NodePort | 80→32086, 443→31015 | Active (older install) |
| `nginx-ingress-v2-ingress-nginx-controller` | NodePort | 80→31184, 443→32662 | **Primary** — all ingresses use class `nginx`, resolved to IP `10.245.201.84` |

All 176 ingress resources use ingress class `nginx` and route to a single load balancer IP: **`10.245.201.84`**.

### Ingress Pattern

Every tenant follows an identical pattern:

```
Host: <tenant>.emrbear.com  →  Service: <tenant>-bear-app:8080
```

**Special ingresses:**

| Host | Namespace | Notes |
|---|---|---|
| `dash.emrbear.com` | `bear-stats` | Internal stats dashboard |
| `bear-deployer.emrbear.com` | `bear-deployer` | Deployment management UI |

| | | |
|---|---|---|
| `services.emrbear.com` | `services` | Shared services API — **only ingress with `force-ssl-redirect: true`** |
| `gemini-staging.emrbear.com` | `gemini` | Gemini staging |
| `demo5.emrbear.com`, `demo.emrbear.com` | `demo5` | Demo (multi-host ingress) |

**TLS:** No TLS secrets are configured directly on ingress resources (0 ingresses with `.spec.tls`). TLS termination is handled upstream (DigitalOcean load balancer or external proxy), with `force-ssl-redirect` enforced only on `services.emrbear.com`.

### Network Policies

Every tenant namespace has exactly one NetworkPolicy: `ingress-network-policy` (selector: `app=bear-app`).

**Rule:** Allow inbound on ports 8080 and 80 **only from** namespaces with labels:

- `name: bear-stats` (monitoring/dashboard scraping)
- `role: ingress` (the nginx-ingress namespace)
- `name: bear-cron` (cron job access)

This effectively isolates tenant pods — no cross-tenant pod-to-pod traffic is permitted. No egress policies are defined (egress is unrestricted).

### Service Type Summary

| Type | Count |
|---|---|
| ClusterIP | 327 |
| NodePort | 2 (both in `nginx-ingress`) |
| LoadBalancer | 0 (handled at cloud level) |
| ExternalName | 0 |

---

## 6. Secrets & Config Handling

### Secret Inventory

| Type | Count |
|---|---|
| `Opaque` | 713 |
| `kubernetes.io/service-account-token` | 208 |
| `kubernetes.io/dockerconfigjson` | 177 |
| `helm.sh/release.v1` | 119 |
| **Total** | **1,217** |

### Cluster-Wide Shared Secrets (replicated to all tenant namespaces)

| Secret Name | Keys | Replicated To | Purpose |
|---|---|---|---|
| `shared-runtime-secrets` | `AWS_IOT_ENDPOINT`, `BEAR_GEMINI_URL`, `SENTRY_DSN` | All ~167 tenant namespaces | Cluster-wide shared runtime env (loaded via `envFrom`) |
| `mariadb-certs` | `ca-cert.pem`, `client-cert.pem`, `client-key.pem` | All ~168 tenant namespaces | TLS certs for MySQL connections |

### Per-Tenant Secret Pattern

Each tenant namespace has two Opaque secrets:

**`<tenant>-bear-app-secrets`** (keys):

- `SECRET_KEY_BASE` — Rails secret key
- `master.key` — Rails credentials master key
- `secrets.yml.key` — Legacy secrets encryption key
- `aws.yml` — AWS configuration
- `mongoid.yml` — MongoDB connection config
- `bear3_production_v1.encrypted_key` + `bear3_production_v1.kekek` — Bear3 symmetric encryption
- `symmetric-encryption.yml` — Symmetric encryption config

**`<tenant>-bear-db-secrets`** (keys):

- `database.yml` — Rails database config (MySQL connection)
- `.my.cnf` — MySQL CLI client config

### How Secrets Are Consumed

| Pattern | Workload | Example |
|---|---|---|
| `envFrom.secretRef` | All tenant `bear-app` and `delayed-*` pods | `shared-runtime-secrets` → all env vars injected |
| `env[].valueFrom.secretKeyRef` | Tenant `bear-app` pods | `SECRET_KEY_BASE` from `<tenant>-bear-app-secrets` |
| Volume mounts (files) | Bear-app pods | `database.yml`, `aws.yml`, `master.key`, cert PEM files mounted as files |

### Infrastructure-Specific Secrets

| Workload | Secret(s) | Keys |
|---|---|---|
| `bear-deployer` | `bear-deployer-secrets` | `REDIS_URL`, `SENTRY_DSN` |
| `bear-services` | `services-db-secrets` | `mongo-password` |
| `bear-xero` | `bear-xero-secrets` | Xero consumer key/secret (NM, DE) |

| | | |
|---|---|---|
| `docker-registry` | `registry-secret` | S3 secret key |
| `redis-bear-deployer-master` | `redis-bear-deployer` | `redis-password` |
| `kube-prometheus-grafana` | `kube-prometheus-grafana` | Grafana admin credentials |
| `prex/cerner-web` | `cerner-secrets`, `prex-aws-secrets`, `prex-postgres-secrets` | Cerner + AWS + Postgres creds |
| `prex/*-surescripts-*` | `surescripts-secrets`, `prex-aws-secrets`, `prex-postgres-secrets` | Surescripts + AWS + Postgres |
| `prex/saaspass-web` | `saaspass-secrets`, `prex-postgres-secrets` | SaaSPass + Postgres |
| `prex/*-idology-web` | `idology-secrets` | IDology credentials |
| `gemini/*` | `gemini-production-secrets`, `gemini-staging-secrets` | Gemini env (separate per environment) |

### ConfigMap Usage

All tenant namespaces have a `<tenant>-bear-app-config` ConfigMap with a single key `custom_hosts` (stores custom domain mappings). Infrastructure ConfigMaps include:

- `redis-bear-deployer` — Redis server config files
- `services-nginx-config` — Nginx config for bear-services
- `xero-config` — Xero redirect URI + scopes

---

## Cross-Check Verification

| Check | Result |
|---|---|
| Every ingress backend service exists in same namespace | ✅ Confirmed (sampled — all follow `<tenant>-bear-app` pattern) |
| `shared-runtime-secrets` exists in all tenant namespaces | ✅ 167 copies found |
| `mariadb-certs` exists in all tenant namespaces | ✅ 168 copies found |
| Secrets referenced by workloads exist | ✅ All verified (bear-deployer-secrets, bear-xero-secrets, services-db-secrets, etc.) |
| Sentry PVCs present but no Sentry pods | ⚠️ Sentry namespace has 3 Redis PVCs (24Gi total) but 0 running pods — likely decommissioned, PVCs orphaned |
| `bear-deployer` scaled to 0 | ℹ️ Both `bear-deployer` and `bear-deployer-sidekiq` deployments have 0 replicas |
| `production-gemini-web` scaled to 0 | ℹ️ Production Gemini web is at 0 replicas (staging is live) |